Del archivo visual al contrato como infraestructura de innovación

Por qué compartir un sistema de diseño desde Figma no basta para automatizar pruebas de concepto y construir frontend con consistencia

Artículo de visión en estado de borrador. Expone la tesis y el modelo de innovación del proyecto; no sustituye sus contratos, decisiones ni especificaciones normativas.

Las organizaciones digitales necesitan convertir ideas en experiencias funcionales con mayor velocidad, pero sin sacrificar coherencia, accesibilidad ni calidad técnica. Durante años, una de las respuestas más efectivas ha sido crear sistemas de diseño: bibliotecas compartidas de componentes, estilos y patrones que permiten dejar de diseñar cada interfaz desde cero.

Ese avance resolvió una parte importante del problema. Sin embargo, no eliminó una fricción más profunda: para convertir un diseño en producto, alguien todavía debe interpretar qué significa.

Un componente puede estar perfectamente construido en Figma y aun así dejar abiertas preguntas esenciales para el frontend. ¿Qué comportamiento es obligatorio? ¿Qué combinaciones son válidas? ¿Cómo responde al teclado? ¿Qué cambia cuando el contenido crece? ¿Cuántos elementos puede contener? ¿Qué token representa una intención semántica y cuál solo coincide visualmente? ¿Qué parte es una regla del sistema y qué parte es apenas un ejemplo?

La tesis de este proyecto es que el siguiente salto no consiste en copiar componentes más rápido ni en perfeccionar el handoff visual. Consiste en trasladar la fuente de verdad a contratos estrictos, versionados y verificables, capaces de gobernar tanto la reconstrucción en Figma como la generación de pruebas de concepto y su posterior traducción a código.

Figma no desaparece. Cambia de papel: deja de ser el único lugar donde intentamos conservar la intención completa del sistema y se convierte en una de sus implementaciones más valiosas.

La necesidad: innovar sin reiniciar la interpretación

Una prueba de concepto debería permitir validar una hipótesis, no volver a negociar desde cero el lenguaje de la interfaz. Sin embargo, en muchos equipos el costo de producir una PoC no está únicamente en escribir código. Está en reconstruir acuerdos dispersos entre pantallas, conversaciones, bibliotecas, documentación, convenciones técnicas y conocimiento tácito.

Ese costo aumenta cuando participan varios equipos, proveedores o agentes de inteligencia artificial. Cada nuevo ejecutor recibe una representación parcial del sistema y completa los vacíos con su propia interpretación. El resultado puede parecer correcto y, al mismo tiempo, apartarse de las reglas que el producto necesitaba preservar.

El problema no es falta de talento. Es falta de una interfaz común entre intención y ejecución.

Para innovar a escala se necesita una gramática estable que pueda ser consumida por personas y herramientas. Debe describir no solo cómo se ve un componente, sino qué es, qué admite, cómo se comporta, qué dependencias utiliza, qué combinaciones prohíbe y cómo se comprueba que una implementación cumple lo acordado.

Cuando esa gramática existe, cada PoC deja de ser una interpretación aislada. Se vuelve una ejecución temporal de una verdad más durable.

El límite de una biblioteca puramente visual

Figma es extraordinario para diseñar, explorar, componer y comunicar visualmente. Una biblioteca bien construida permite compartir componentes, variantes, propiedades, variables y patrones. Pero una biblioteca visual no contiene automáticamente todas las decisiones necesarias para construir software.

La apariencia puede demostrar que existe un estado llamado **disabled**; no demuestra por sí sola si el elemento debe excluirse del orden de tabulación, bloquear eventos o anunciarse de una manera específica a tecnologías de asistencia. Un frame puede mostrar una altura; no demuestra si esa altura es fija, mínima o resultado del contenido. Una lista con tres elementos no establece que tres sea su límite. Dos colores idénticos no prueban que compartan el mismo significado semántico.

En otras palabras, Figma muestra una solución. El frontend necesita también las reglas que permiten reproducirla y adaptarla sin romper su intención.

La brecha aparece porque diseño y código operan con modelos diferentes. En diseño se manipulan capas, instancias, variantes y propiedades. En frontend se definen APIs, estados de ejecución, eventos, estructuras de datos, restricciones de contenido, semántica HTML, comportamiento responsivo y pruebas. Algunas correspondencias son directas; muchas otras requieren una decisión explícita.

Si esa decisión no está documentada como parte de la fuente de verdad, reaparece en cada implementación.

Las soluciones que parecen suficientes, pero no resuelven el fondo

Copiar componentes entre archivos

Copiar acelera el acceso a una forma visual, pero también copia una fotografía del sistema. No conserva necesariamente su procedencia, sus dependencias, su evolución ni el motivo detrás de cada decisión. Con el tiempo aparecen variantes locales, correcciones inconexas y versiones visualmente semejantes que ya no significan lo mismo.

La organización distribuye componentes, pero también distribuye la posibilidad de que cada copia adquiera una verdad distinta.

Publicar y compartir una librería de Figma

Una biblioteca publicada mejora el control y reduce la duplicación. Es una pieza indispensable de una operación de diseño madura. Aun así, compartirla no transforma automáticamente sus propiedades en contratos de software.

El consumidor puede descubrir qué variantes existen, pero no necesariamente por qué existen, cuáles pueden combinarse, qué estados dependen del producto, cómo se resuelven

datos incompletos o qué criterios determinan que el componente está correctamente implementado fuera de Figma.

La librería centraliza el activo visual; no siempre centraliza la decisión.

Añadir más documentación de handoff

Las anotaciones, especificaciones y páginas de uso reducen ambigüedad, pero con frecuencia siguen separadas del artefacto que termina gobernando la implementación. Si una regla cambia en un documento y no en los demás, el equipo vuelve a elegir entre fuentes competidoras.

Documentar más no basta. La documentación debe establecer autoridad, versiones, procedencia y criterios verificables. También debe distinguir entre dato canónico, observación, inferencia y decisión pendiente.

Convertir el diseño directamente a código

Los generadores visuales pueden acelerar la creación de estructura y estilos. Su límite aparece cuando se espera que deduzcan intención a partir de la superficie. Una herramienta puede traducir un auto layout a CSS, pero no decidir legítimamente la API pública del componente, la semántica accesible, las reglas de negocio o las combinaciones válidas si esas decisiones nunca fueron expresadas.

La traducción directa suele optimizar el último tramo —de una representación visual a una representación técnica— sin resolver la ausencia de especificación entre ambas.

Convertir al equipo central en intérprete permanente

Otra respuesta común es concentrar el conocimiento en un equipo de sistema de diseño que revisa cada caso. Esto protege la consistencia, pero puede convertir la gobernanza en un cuello de botella. La organización obtiene control a cambio de depender de unas cuantas personas para traducir intención una y otra vez.

El objetivo no debería ser eliminar la gobernanza, sino codificarla lo suficiente para que la ejecución pueda distribuirse sin perderla.

Dónde se pierde la intención al construir frontend

La distancia entre una librería visual y una implementación productiva no es un único hueco. Es una cadena de pequeñas decisiones que, si permanecen implícitas, multiplican la divergencia.

Evidencia disponible en diseño	Pregunta que sigue abierta	Lo que necesita la implementación
Variante o propiedad visual	¿Representa una opción pública, una condición interna o solo una organización del archivo?	Una API conceptual con nombres, tipos, valores válidos y restricciones.
Estado como **hover**, **disabled** o **loading**	¿Qué eventos, foco, anuncios y transiciones implica?	Reglas de comportamiento, accesibilidad y manejo de eventos.
Variable o color resuelto	¿Cuál es su rol semántico, su alias y su alcance?	Tokens trazables; no literales elegidos solo por igualdad visual.
Auto layout y dimensiones observadas	¿Qué ocurre con contenido largo, pantallas pequeñas o localización?	Reglas responsivas, límites, overflow y comportamiento de contenido.
Tres elementos colocados en una región	¿Tres es una muestra o una cardinalidad fija?	Reglas de composición, slots, contenido permitido y cantidades válidas.
Prototipo con una interacción	¿Es una demostración visual o una regla completa de producto?	Estados de ejecución, condiciones, resultados, errores y casos límite.
Pantalla visualmente correcta	¿Qué significa cumplir y cómo se evita una regresión?	Criterios de aceptación y pruebas vinculadas a requisitos.

Estos gaps explican por qué dos equipos competentes pueden partir del mismo archivo y producir componentes distintos. No están leyendo mal la interfaz; están completando de forma diferente aquello que nunca fue explicitado.

También explican por qué un prototipo generado automáticamente puede impresionar en una demostración y fallar al entrar en un flujo real. La generación visual produce una superficie. Un producto necesita comportamiento, datos, restricciones y evidencia de cumplimiento.

La propuesta: contratos como fuente de verdad estricta

Un contrato de componente no es una captura enriquecida ni una descripción informal. Es una especificación legible por personas y suficientemente determinista para orientar herramientas. Define la identidad del componente, su anatomía, propiedades, variantes, tamaños, estados, comportamiento, accesibilidad, tokens, contenido, composición, responsividad, casos límite, combinaciones prohibidas, mapeo entre diseño y código y criterios de aceptación.

Cuando un dato ya tiene una fuente canónica —por ejemplo, un inventario de tokens— el contrato no lo duplica como una segunda lista editable. Lo referencia. El Markdown gobierna las reglas; los datos estructurados gobiernan los campos que el contrato les delega. La trazabilidad conecta ambos con sus implementaciones.

Esta separación es importante porque evita una falsa precisión. Un valor puede estar definido exactamente en JSON mientras su intención de uso se expresa en el contrato. Un componente puede observarse en Figma sin convertir automáticamente cada detalle encontrado en una regla aprobada. Una pregunta sin evidencia puede permanecer como **UNRESOLVED** en lugar de ser rellenada por intuición.

El contrato se convierte así en una interfaz entre intención y destinos:

```
Fuentes y decisiones
□
Contrato versionado + datos canónicos
□
Figma · PoC · frontend · documentación · QA
```

La dirección de la dependencia cambia. Ya no se intenta extraer toda la verdad desde el resultado visual. Se construyen resultados verificables desde una verdad explícita.

De documento a capacidad generativa

El valor más ambicioso del contrato aparece cuando deja de ser únicamente material de consulta y se convierte en entrada operativa.

Si propiedades, estados, tokens, composición y criterios de aceptación están expresados de forma consistente, pueden alimentar adaptadores y generadores. Una herramienta puede producir la base de una prueba de concepto, preparar una API de componente, vincular tokens, generar casos de prueba o detectar combinaciones inválidas sin tener que inferirlas de píxeles y nombres de capas.

Esto no significa que cualquier contrato actual genere automáticamente código listo para producción. Esa es la arquitectura objetivo, no una capacidad que deba darse por realizada. Para alcanzarla serán necesarios esquemas estables, validadores, mapeos por plataforma y una cadena de QA que compare cada salida con el contrato vigente.

Pero el cambio fundamental ocurre antes de construir esos generadores: por primera vez existe una entrada diseñada para ser interpretada de forma consistente tanto por una persona como por una máquina.

La prueba de concepto deja de depender de la memoria del equipo y puede ensamblarse desde reglas conocidas. Si el experimento prospera, parte de esa estructura puede traducirse limpiamente a React, TypeScript u otra plataforma. Si se descarta, el conocimiento valioso no desaparece con el prototipo: permanece en el contrato.

Descentralizar la construcción sin descentralizar la verdad

La descentralización suele asociarse con pérdida de control. En este modelo ocurre lo contrario: la ejecución se distribuye porque la autoridad se vuelve más clara.

Diseñadores, desarrolladores, equipos de producto, proveedores y agentes pueden trabajar sobre el mismo contrato sin necesitar acceso permanente al intérprete original. Cada actor conserva libertad para usar la herramienta adecuada a su destino, pero las decisiones compartidas —nombres, estados, tokens, restricciones y criterios de aceptación— no cambian silenciosamente en cada transferencia.

La gobernanza permanece centralizada en el sentido correcto: existe una fuente normativa, un proceso de revisión, versiones, procedencia y manejo explícito de conflictos. La producción se descentraliza: más equipos pueden construir, validar y experimentar en paralelo.

No se trata de permitir que cualquiera invente una variante. Se trata de permitir que cualquiera implemente con confianza aquello que ya fue decidido, y que cualquier nueva decisión vuelva al sistema de manera trazable.

Qué tiene de disruptiva esta idea

Lo disruptivo no es usar Markdown, JSON o inteligencia artificial. Tampoco es generar código desde diseño, una promesa que la industria ha perseguido durante años. El cambio está en mover el centro de gravedad del sistema.

Primero, se pasa de reutilizar artefactos a reutilizar decisiones. Una instancia copiada conserva una forma; un contrato conserva las condiciones que permiten producir muchas implementaciones coherentes.

Segundo, el sistema de diseño deja de ser únicamente un catálogo de componentes y se convierte en infraestructura de producto. Puede gobernar diseño, prototipado, código, documentación y QA desde una base común.

Tercero, el prototipo cambia de naturaleza. Ya no es una maqueta aislada que deberá reinterpretarse si funciona. Es una ejecución reemplazable de una especificación durable. El aprendizaje puede modificar el contrato y propagarse a los siguientes destinos.

Cuarto, las herramientas se vuelven sustituibles. Figma, un framework de frontend o un generador concreto pueden evolucionar sin obligar a que la intención del sistema viva encerrada en uno de ellos. La fuente de verdad permanece; cambian sus compilaciones.

Finalmente, la inteligencia artificial obtiene un contexto mucho más confiable. En lugar de pedirle que adivine un sistema observando pantallas, se le entrega una especificación con límites, dependencias y criterios de aceptación. La automatización deja de basarse principalmente en semejanza visual y comienza a basarse en reglas.

Lo que este enfoque no promete

Un contrato no sustituye el criterio de diseño, la ingeniería ni la investigación de producto. Tampoco elimina la necesidad de revisar accesibilidad, rendimiento o experiencia real. No convierte toda pantalla en código de producción con un clic y no garantiza que dos plataformas deban tener implementaciones idénticas.

La precisión tiene un costo: los contratos deben mantenerse, versionarse y validarse. Las contradicciones deben detener la parte afectada en lugar de resolverse en silencio. Las decisiones nuevas deben regresar a la fuente de verdad. Una automatización sin gobernanza solo produce inconsistencias más rápido.

La propuesta, por tanto, no es reemplazar a quienes construyen. Es liberarles de reinterpretar repetidamente decisiones que la organización ya tomó y permitirles concentrarse en los problemas que sí requieren juicio.

Cómo demostrar el valor

La idea puede validarse con un piloto acotado, no con una promesa general. Conviene elegir una familia de componentes que combine estados, tokens, contenido variable, composición y comportamiento. El experimento debería producir desde el mismo contrato una reconstrucción verificable en Figma y una prueba de concepto funcional en frontend.

La comparación con el flujo visual tradicional puede observar:

- tiempo desde la definición hasta una PoC utilizable;
- número de decisiones que requieren aclaración durante la implementación;
- divergencias entre diseño, código y documentación;
- retrabajo provocado por estados, contenido o combinaciones no contempladas;
- cobertura de requisitos y trazabilidad de cada resultado;
- facilidad para que un segundo equipo reproduzca la implementación sin recurrir al autor original.

No se necesita demostrar que la automatización reemplaza todo el proceso. Basta con probar que una fuente estricta reduce interpretación, conserva intención y permite generar resultados comparables en más de un destino.

Una nueva forma de pensar el sistema de diseño

Compartir componentes seguirá siendo útil. Publicar una biblioteca seguirá siendo necesario. Diseñar en Figma seguirá siendo una parte central del trabajo. La diferencia es reconocer que ninguna de esas acciones, por sí sola, constituye una fuente completa de verdad para el software.

El sistema de diseño que habilita la siguiente generación de productos no será solamente el lugar donde viven los componentes. Será el lugar donde viven las decisiones que permiten construirlos.

Al convertir esas decisiones en contratos estrictos, la organización puede automatizar pruebas de concepto, traducir intención a código con menos pérdida y distribuir la ejecución entre más

personas y herramientas. No elimina la interpretación creativa donde aporta valor; elimina la interpretación accidental donde genera deuda.

Ahí reside la oportunidad: pasar de entregar imágenes para ser interpretadas a compartir una verdad que pueda ser ejecutada.

Fundamento dentro del proyecto

Este artículo sintetiza la dirección expresada por el método Specs Driven Design del repositorio y, en particular, por los protocolos de síntesis, trazabilidad, QA y revisión de Slots. Las afirmaciones sobre automatización de PoC y generación de frontend describen la visión y la arquitectura objetivo; no declaran que esa cadena esté implementada o autorizan por sí mismas la construcción del frontend.

- AGENTS.md
- P03 · Construcción de contratos
- P05 · Autodocumentación y trazabilidad
- P06 · QA y validación
- P-SLOTS · Revisión e implementación de Slots